



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Create Short Video Film using AI

TGS Ref No: TGS-2020505925

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 12.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
10.0	Legacy release (superseded)	Legacy Learner Guide for 'Image and Video Processing with OpenCV'.	Dr. Alfred Ang
11.0	6 September 2026	Rebuilt as 'Generative AI for Image and Video Creation' with twelve computer-vision and generative labs. Superseded by 12.0.	Dr. Alfred Ang
12.0	27 September 2026	Retitled 'Create Short Video Film using AI' and revamped for AI short-film production. Twelve new labs with full numbered procedures build one 48-second film, The Last Ferry, from need analysis to the final export ladder, each with its own data, SIMULATED media, prompt pack, expected outputs, evidence checklist and troubleshooting. New glossary of film-production and AI-video terms; source register extended with seven current AI short-film tutorials (S6-S12). Learning outcomes, K1-K13, A1-A8 and assessment structure unchanged. Slide references regenerated against the 138-slide v12.0 deck.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction.....	4
How to Use This Guide.....	4
Course Learning Outcomes.....	4
Assessed Criteria in Full.....	5
Before You Start — Environment Setup.....	6
Provenance Rules That Apply to Every Artefact.....	7
Topic 01 — AI Vision and Generative AI for Short Film Creation.....	7
Lab 01 — Film Needs Analysis and AI Production Pipeline Specification.....	8
Topic 02 — AI Image Processing, Generation and Visual Enhancement.....	11
Lab 02 — Frames, Formats and Aspect Ratios for Film Delivery.....	12
Lab 03 — Structured Prompting for Cinematic Stills.....	14
Lab 04 — State Variants, Clean-Up and Reframing with Masks.....	16
Lab 05 — Cinematic Grading and Enhancement Measured with PSNR and SSIM.....	19
Topic 03 — AI Visual Features, Character Consistency and Scene Design.....	21
Lab 06 — Character Reference Sheets and Consistency Features.....	22
Lab 07 — Look Bible and Scene Continuity with Global Descriptors.....	24
Topic 04 — AI Video Generation, Animation and Video Analytics.....	26
Lab 08 — Cast-Presence Detection and Detector Evaluation.....	27
Lab 09 — Storyboard to Shot Prompts, Start and End Frames, and the Animatic.....	30
Lab 10 — Shot, Motion and Continuity Analytics on the Assembled Sequence.....	32
Lab 11 — Voiceover, Music, Captions and the Final Export Ladder.....	34
Topic 05 — Cloud and Edge AI Workflows for Short Film Production.....	37
Lab 12 — Cloud, Local and Hybrid Production Architecture Evaluation.....	37
Preparing for the Assessment.....	40
Glossary.....	41
Source Register.....	42

Introduction

This Learner Guide accompanies the WSQ course Create Short Video Film using AI (TGS-2020505925), conducted by Tertiary Infotech Academy Pte Ltd. It is version v12.0, released 27 September 2026.

The course is delivered over two days across five topics, and is assessed against the Skills Framework TSC Computer Vision Technology (ICT-DIT-4022-1.1) at Proficiency Level 4. The Written Assessment covers the thirteen knowledge statements K1 to K13; the Practical Performance covers the eight ability statements A1 to A8.

The slides carry the mechanisms, the measurements and the decision rules. This guide carries the FULL numbered procedure for every lab, so you can repeat any lab after the course without the trainer present. Both are open-book references in the assessment.

Note: Every lab builds part of one 48-second short film, The Last Ferry, and runs on the base `opencv-python` wheel plus NumPy with no paid service and no downloaded model. AI image, video, voice and music tools are optional extensions: each lab supplies SIMULATED media so the deliverable can always be completed offline.

How to Use This Guide

- Read the topic section before its labs — the guide's topic sections state the mechanism the labs then measure.
- Work the labs in order. They follow the production of one film — plan, references, stills, shots, analytics, sound and export — and Lab 12 costs the whole pipeline.
- Every lab folder under labs/ carries the same README as the steps reproduced here, plus its data, its media, its prompt pack and a PDF of every Markdown file.
- Complete the evidence checklist at the end of each lab. Those items are what an assessor looks at.
- Where a figure appears, check its label: MEASURED, MODELLED, PUBLISHED or ILLUSTRATIVE. Never lift an ILLUSTRATIVE figure into a client document.

Course Learning Outcomes

- LO1: Understand basic vision systems concepts and applications
- LO2: Apply image processing
- LO3: Implement feature extraction
- LO4: Apply machine learning based computer vision methods
- LO5: Implement video analytics algorithms
- LO6: Evaluate edge vs cloud-based computer vision systems

ELO	Statement	Abilities	Knowledge	Delivery topic
ELO1	Understand basic vision systems concepts and applications	A1	K1, K2	Topic 01
ELO2	Apply image processing	A2	K3, K4	Topic 02
ELO3	Implement feature extraction	A3, A4	K5, K6, K7	Topic 03
ELO4	Apply machine learning based computer vision methods	A5	K8, K9	Topic 04
ELO5	Implement video analytics algorithms	A6	K10	Topic 04
ELO6	Evaluate edge vs cloud-based computer vision systems	A7, A8	K11, K12, K13	Topic 05

Assessed Criteria in Full

Knowledge — assessed in the Written Assessment (13 questions, 60 minutes)

Code	Knowledge statement	Question
K1	Vision system concepts	Question 1
K2	Business applications of vision systems	Question 2
K3	Methods to represent image and video data	Question 3
K4	Image and video processing, filtering and transformation methods	Question 4
K5	Feature extraction and representation techniques	Question 5
K6	Local feature descriptions, edge, colour, texture and motion	Question 6
K7	Global feature descriptions, statistical and geometrical methods	Question 7
K8	Deep learning concepts	Question 8
K9	Object segmentation, detection and recognition	Question 9
K10	Activity tracking, generative models, scene understanding and event discovery	Question 10
K11	Vision system architecture	Question 11
K12	Vision communication protocols	Question 12
K13	Real-world design constraints and solution options	Question 13

Abilities — assessed in the Practical Performance (5 tasks, 90 minutes)

Code	Ability statement	Task	Practised in
A1	Identify the needs of vision systems technology in industrial applications	Task 1	Lab 01
A2	Apply the principles of processing, filtering and analysis methods for video data	Task 2	Lab 02, Lab 03, Lab 04, Lab 05
A3	Analyse global feature descriptions	Task 2	Lab 07
A4	Design and implement feature extraction and representation methods	Task 3	Lab 06
A5	Design and apply machine-learning based methods for object detection, object tracking and activity recognition	Task 4	Lab 08, Lab 09, Lab 11
A6	Design and apply video analytics algorithms for high-level video analytics tasks	Task 4	Lab 09, Lab 10, Lab 11
A7	Design the architecture of applied vision systems	Task 5	Lab 12
A8	Design, develop and evaluate edge-based and cloud-based systems	Task 5	Lab 12

Note: Nothing is assessed that is not taught. Every question traces to a slide range and every task traces to a lab; the answer keys cite both.

Before You Start — Environment Setup

What you need

- A Windows or macOS laptop with Python 3.10 or later.
- opencv-python 4.10 or later, and NumPy. Nothing else is required.
- Optional: FFmpeg (Lab 11 uses it to mux audio into the final export).
- Optional: accounts on AI image, video, voice and music tools for the generation extensions. Record the model ID, date and prompt of everything you keep.

Install once, before the class if you can

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Tool and API currency

- AI video tools change month to month. Tools named in this guide are examples of a technique drawn from the source tutorials (S6–S12), not endorsements. Check current features, clip limits, prices and terms before relying on one.
- The `cv2.quality` SSIM module is contrib-only. The course ships its own NumPy SSIM in `ssim.py` rather than adding a dependency for one function.
- `cv2.data.harcascades` and `cv2.HOGDescriptor_getDefaultPeopleDetector()` ship trained detectors inside the package, so Lab 08 runs two machine-learning detectors with no download.

Conventions used in every lab

- Run commands from inside the lab folder, not from its parent.
- Each lab writes everything it produces into its own `out/` directory.
- `cv2.imread` returns `None` on failure and does NOT raise — always guard it.
- Array indexing is `[row, column] = [y, x]`; drawing functions take `(x, y)`.
- Masks: WHITE = edit, BLACK = preserve. State the convention whenever you hand a mask to a tool.

Provenance Rules That Apply to Every Artefact

1. A **SIMULATED** asset is a deterministic OpenCV illustration of The Last Ferry made for this course. It is never described as a photograph or as generative-model output. Every such file carries a burned-in banner.
2. An **AI-GENERATED** reference is kept unchanged with a record of the tool, model ID, date and exact prompt. Many generators also embed a watermark such as SynthID or C2PA Content Credentials — do not strip them.
3. An **ANIMATIC** is storyboard keyframes timed to the voiceover, labelled on every frame. It fixes timing before credits are spent and is never submitted as a generated film.
4. The label travels to every derivative. A crop, reframe or grade of a simulated asset is still a simulated asset, and the export re-burns the label.
5. A film you publish that contains AI-generated material is disclosed as such, on the platform's AI-content label and in the credits.

Note: Mislabelling provenance is an assessment failure and, in production, a governance breach. This is the one rule in the course with no exceptions.

Topic 01 — AI Vision and Generative AI for Short Film Creation

LU1 · ELO1 · knowledge K1, K2 · abilities A1

Slides 25–43 · 75 minutes classroom facilitation · 0 minutes practical

The AI film pipeline as a vision system · how image and video generators work — diffusion, diffusion transformers, VAEs and GANs · text-to-image, image-to-video and text-to-video · reference-first production · business applications and the need test

Key concepts

An AI film pipeline is a vision system with a synthesis stage. Sense (scripts, references, footage) -> represent (frames, latents, formats) -> analyse (continuity and quality measurements) -> decide (approve, regenerate, re-edit) -> synthesise (generate the next shot). Generation adds the last stage; it does not remove the first four.

Generators model p(frames | condition). Text-to-image, image-to-video and text-to-video differ only in the condition. The more the condition fixes — a character sheet, a location plate, a start frame — the less the model has to invent, and the less it drifts.

Today's video models are diffusion transformers. A VAE compresses frames to a latent; a transformer denoises spatiotemporal patches over S steps, guided by text and image conditions. GANs survive mainly as fast upscalers and face restorers. Each family fails in a recognisable way.

Reference-first beats prompt-first. Beginners send a story straight to a video model and reroll for hours. Professionals build character, location and prop references first, so the video model has nothing left to guess. Most of the quality is decided before the first clip is generated.

Business value comes from repeatable short-form storytelling. Brand films, product teasers, training micro-dramas, social series, pitch previsualisation and event recaps. Value comes from volume, speed to first cut and the ability to iterate a story without re-shooting.

The need test decides the workflow, not the hype. Score each film idea on volume, repeatability, turnaround, review tolerance, likeness/rights risk and cost gap. Then choose: an all-in-one agent (fast, less control), a multi-tool pipeline (control, consistency) or a live shoot.

Lab 01 — Film Needs Analysis and AI Production Pipeline Specification

Outcome: ELO1 · **Abilities:** A1 · **Knowledge:** K1, K2

Slide: 43 · **Folder:** [labs/lab-01-film-needs-analysis/](#) · **In class:** facilitated within LU1 classroom facilitation (no separate practical minutes)

Scenario

Harbourline Ferries (a fictional ferry operator) turns 50. Six departments have each asked the studio for 'AI video'. You are the production lead: score the six film ideas against measurable need criteria, apply the likeness, rights and authenticity gates, choose a workflow pattern for each project you select, and specify the five-stage

production pipeline for the anniversary film The Last Ferry — the film you will build in every later lab.

What you will produce

A scored shortlist with a written rationale for every rejected or held project, a workflow-pattern choice per selected project, and a five-stage pipeline specification for The Last Ferry listing the references it needs.

Tools: Spreadsheet or text editor · Python 3 (optional baseline scorer) · no paid service required

Files supplied with this lab

Path	What it is
reference/key-art-concept.png	Concept key art for The Last Ferry (SIMULATED flat illustration, not AI output)
brief/studio-brief.md	The client brief with four hard constraints
data/candidate-projects.csv	Six candidate film projects with volume, turnaround, review, likeness, cost and rights fields
data/need-criteria.json	Weights, scoring rules, the three gates, the accept threshold and the three workflow patterns
score_projects.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Create an `out/` folder inside this lab folder. Open `brief/studio-brief.md` and write down the four hard constraints in your own words: likeness and consent, rights, authenticity, and the 30% cost-reduction floor.

2. Open `data/candidate-projects.csv`. Confirm there are six projects and read every column, including `notes`. Nothing is scored yet.

```
python3 -c "import csv;r=list(csv.DictReader(open('data/candidate-projects.csv')));print(len(r), list(r[0]))"
```

3. Open `data/need-criteria.json`. Note the six criteria (volume, repeatability, speed_value, review_tolerance, likeness_risk, cost_gap), their weights (which sum to 1.0), the accept threshold of 3.6 and the three gates.

4. Create `out/manual-scores.csv` with the headers `project_id`, `volume`, `repeatability`, `speed_value`, `review_tolerance`, `likeness_risk`, `cost_gap`, `weighted_total`, `verdict`. Score P1 (The Last Ferry) BY HAND first: finished minutes per quarter = episodes × seconds ÷ 60 = 7 × 30 ÷ 60 = 3.5 min, which scores 3 on volume.

5. Finish P1: cost gap = (38000 – 9000) ÷ 38000 = 0.76, which scores 5. Multiply each score by its weight and add them. You should reach 3.9. Show the arithmetic in the file.

6. Score the other five projects by hand in the same way. Then run the baseline scorer and compare. Reconcile every difference in one sentence — do not change a score just to match the baseline.

```
python3 score_projects.py --csv data/candidate-projects.csv --criteria
data/need-criteria.json --out out
```

7. Apply the gates BEFORE the threshold. P4 scores 3.8 but uses passenger faces without consent: REJECT. P5 is a factual statement by a real executive: AI is not appropriate, REJECT. P3 scores 4.1 but regulator approval is **pending**: HOLD, and name the evidence that would release it.

8. Write `out/decision.md`. For every rejected or held project, give one sentence that names the failed gate or criterion AND the number, e.g. 'P4 — likeness gate FAIL: 400 auto-sent videos a quarter built from unconsented passenger selfies'.

9. For each selected project (P1, P2, P6), choose a workflow pattern from `workflow_patterns` in the criteria file — all-in-one agent, multi-tool pipeline or hybrid/local — and justify it in one line using volume, control and review needs.

10. Specify the five-stage pipeline for The Last Ferry in `out/pipeline-spec.md`. SENSE: the inputs (brief, script, cast/location/prop references, voiceover). REPRESENT: master format 16:9, 24 fps, 1920×1080, plus the 9:16 cut-down.

11. Continue the specification. ANALYSE: the measurements every shot must pass (character consistency, look match, cast count, continuity). DECIDE: the approve / regenerate / re-edit rule and who signs off. SYNTHESISE: which generators produce stills, clips, voice and music.

12. List the references the film needs before any video is generated: two character sheets (MEI, CAPTAIN TAN), two locations (NIGHT MARKET, HARBOUR PIER in two states) and one prop (RED BOX). Explain in one sentence why reference-first reduces rerolls.

13. Optional extension (outside class time): paste the brief into an LLM with the planning prompt in `PROMPTS.md` and compare its shortlist with yours. Record where it ignored a gate.

Verify

Your shortlist selects P1, P2 and P6; holds P3 on an UNKNOWN gate; rejects P4 and P5 on gates; and every rejection names the criterion or gate and the number.

Expected outputs

- `out/manual-scores.csv` with six rows and your arithmetic for P1
- `out/baseline-scores.csv` from the scorer, reconciled against your scores
- `out/decision.md` with a rationale for P3, P4 and P5 and a workflow pattern for P1, P2 and P6

- out/pipeline-spec.md with five stages and the reference list for The Last Ferry

Evidence checklist

- Manual score sheet with the P1 calculation shown
- Written reconciliation of any difference with the baseline scorer
- One-sentence gate or criterion rationale per rejected or held project
- Workflow-pattern choice with a justification for each selected project
- Five-stage pipeline specification and reference list for The Last Ferry

Troubleshooting

Symptom	What to do
My total differs from the baseline by 0.1–0.3	Check the band edges: 3.5 minutes is volume 3 (3–<10), a 0.3 cost gap is 3 not 1.
P4 passes on score — should I select it?	No. Gates are applied first; a failed likeness gate rejects the project whatever the score.
P3 has the second-highest score	It is HOLD, not SELECT: the regulator gate is UNKNOWN. Name the evidence needed.
python3 is not installed	Complete the lab by hand in a spreadsheet; the scorer is only a baseline.

Note: The same content, plus the full prompt pack, is in [labs/lab-01-film-needs-analysis/README.md](#) and its PDF. Sources used in this lab: S5, S6, S7, S9, S12.

Topic 02 — AI Image Processing, Generation and Visual Enhancement

LU2 · ELO2 · knowledge K3, K4 · abilities A2

Slides 44–66 · 75 minutes classroom facilitation · 60 minutes practical

Frames, resolution, frame rate, aspect ratio and bitrate · structured prompting for cinematic stills · state variants, inpainting and outpainting for reframing · colour grading, denoising, sharpening and upscaling measured with PSNR and SSIM

Key concepts

A frame is an H x W x C array; a film is frames plus a clock. OpenCV returns H x W x 3 uint8 in BGR order. A 1920x1080 RGB stream at 24 fps is $1920 \times 1080 \times 3 \times 24 = 149.3$ MB/s uncompressed; a delivery codec brings that to a few MB/s. Frame count / fps = duration.

Aspect ratio is a delivery decision made before generation. 16:9 for YouTube and screens, 9:16 for Reels, Shorts and TikTok, 4:5 or 1:1 for feeds. A 9:16 centre crop of a 16:9 frame keeps only 31.6% of its width, so decide the master format first and keep action inside the shared safe area.

Structured prompts reduce variance. Six fields — subject, action, environment, camera, style, constraints — or the same fields as JSON. Lock the fields that must not change, vary one factor at a time, and explore on a fast, cheap model before committing credits.

An edit is masked, conditioned regeneration. State variants ('same pier, ferry pulling away — change nothing else'), inpainting to remove a stray object, outpainting to extend 16:9 to 9:16. White = edit, black = preserve in these labs. Always measure the pixels outside the mask: a good-looking edit can still move the background.

Grading is a point operation; filtering is a neighbourhood operation. White balance, curves and a teal-and-orange LUT map each pixel on its own. Blur, denoise and unsharp masking combine neighbouring pixels with a kernel. Resizing and reframing are geometric transforms (affine / perspective).

Enhancement must be measured, not admired. PSNR rewards per-pixel accuracy and forgives blur; SSIM rewards preserved structure. Upscale only the takes you keep, and check for halos and plastic skin before an enhancement reaches the timeline.

Lab 02 — Frames, Formats and Aspect Ratios for Film Delivery

Outcome: ELO2 · **Abilities:** A2 · **Knowledge:** K3

Slide: 63 · **Folder:** [labs/lab-02-frames-formats-aspect-ratios/](#) · **In class:** 15 minutes

Scenario

Before any shot of The Last Ferry is generated, the team must agree the master format and the delivery ladder. You inspect a keyframe and the 12-second rough cut the way the machine sees them — arrays, channel order, frame rate, duration and bitrate — and measure what a 9:16, 4:5 or 1:1 reframe keeps of the 16:9 master.

What you will produce

A clip report (shape, dtype, BGR/RGB/HSV of the calibration chips, fps, frame count, duration, bitrate, compression ratio), a contact sheet, and a reframe per delivery format with the percentage of the master each one keeps.

Tools: Python 3 · opencv-python · NumPy · no network

Files supplied with this lab

Path	What it is
reference/keyframe-shot2.png	Keyframe of shot S2 with four exact colour chips bottom-left (SIMULATED)
reference/rough-cut.mp4	12-second 960x540 24 fps rough cut: three shots (SIMULATED clip)
data/delivery-specs.json	Master and four delivery formats with UI safe areas
inspect_clip.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Install the environment once (see Environment) and run everything from inside this lab folder.

2. Load the keyframe and print its shape and dtype. Expect (540, 960, 3) uint8: height first, then width, then three channels.

```
python3 -c "import cv2;im=cv2.imread('reference/keyframe-shot2.png');print(im.shape, im.dtype)"
```

3. `cv2.imread` returns None on a bad path instead of raising — guard it. Try a wrong filename and confirm you get None, not an exception.

```
python3 -c "import cv2;print(cv2.imread('reference/nope.png'))"
```

4. Read the first colour chip at `x=21, y=515`. Arrays are indexed [row, column] = [y, x]. The chip is pure blue: OpenCV reports BGR [255, 0, 0].

```
python3 -c "import cv2;im=cv2.imread('reference/keyframe-shot2.png');print(im[515,21])"
```

5. Run the inspector. It prints each chip in BGR, RGB and HSV, measures the clip and writes the delivery reframes.

```
python3 inspect_clip.py --still reference/keyframe-shot2.png --clip reference/rough-cut.mp4 --specs data/delivery-specs.json --out out
```

6. Check the clock: 288 frames ÷ 24 fps = 12.0 s. Compare `frames_declared` with `frames_counted` — containers can misreport, so count when it matters.

7. Compute the uncompressed rate by hand: $960 \times 540 \times 3 \times 24 = 37.3$ MB/s. At the production master of 1920×1080 it is 149.3 MB/s. Compare with the file's bitrate and note the compression ratio.

8. Open `out/contact-sheet.png` (one frame per second). Identify the three shots and the seconds where they cut.

9. Open each `out/reframe_*.png`. The yellow rectangle is the platform UI safe area. Record the percentage of the master each format keeps: 9:16 keeps about 31.7%.

10. Decide: will The Last Ferry be generated natively in 16:9 and cut down, or will the vertical version be generated natively? Write the rule in `out/format-decision.md` with the numbers.

11. Write the blocking rule for every shot prompt: keep faces, the RED BOX and all captions inside the centre third of a 16:9 frame so every cut-down keeps them.

Verify

The chip at (21, 515) reads BGR [255, 0, 0]; the clip reports 24 fps, 288 frames and 12.0 s; the 9:16 reframe keeps about 31.7% of the master.

Expected outputs

- out/clip-report.json
- out/contact-sheet.png
- out/reframe_youtube_16x9.png, reframe_reels_9x16.png, reframe_feed_4x5.png, reframe_square_1x1.png
- out/format-decision.md

Evidence checklist

- Printed shape, dtype and the BGR/RGB/HSV values of all four chips
- Hand calculation of the uncompressed data rate and the measured compression ratio
- Table of the percentage of the master each delivery format keeps
- Written master-format and blocking decision for The Last Ferry

Troubleshooting

Symptom	What to do
imread returns None	Run from inside the lab folder or give the full path; check the extension.
Colours look swapped in matplotlib	OpenCV is BGR; convert with <code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code> before plotting.
frames_declared differs from frames_counted	Trust the counted value; some containers store an estimate.
VideoCapture cannot open the mp4	Reinstall opencv-python (it bundles FFmpeg); do not use opencv-python-headless without FFmpeg.

Note: The same content, plus the full prompt pack, is in [labs/lab-02-frames-formats-aspect-ratios/README.md](#) and its PDF. Sources used in this lab: S5, S15, S16, S17.

Lab 03 — Structured Prompting for Cinematic Stills

Outcome: ELO2 · **Abilities:** A2 · **Knowledge:** K3, K4

Slide: 64 · **Folder:** [labs/lab-03-structured-prompting-stills/](#) · **In class:** 15 minutes

Scenario

Shot S2 — MEI runs through the night market — keeps coming back different: the coat changes colour, she jumps around the frame, the look shifts. You lint a weak six-field prompt, rewrite it, and measure four prompt designs (A: free prose → D: subject, look, camera and constraints locked) on four takes each.

What you will produce

A linted v1 prompt with its problems listed, your corrected v2 prompt in prose and JSON, and a variance table showing how much each prompt design reduced wardrobe, blocking, framing and look drift.

Tools: Python 3 · opencv-python · NumPy · optional: any image generator for your own takes

Files supplied with this lab

Path	What it is
reference/takes/cellA_01.png ... cellD_04.png	16 takes of shot S2: four prompt designs × four takes (SIMULATED, designed spread)
data/shot2-fields-v1.json	A weak six-field prompt with deliberate problems
data/shot2-fields-v2-template.json	The v2 template with the subject and references filled in
data/prompt-factors.json	What each design A-D locks, and what each measure means
prompt_lab.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Read `data/prompt-factors.json`: the six fields are subject, action, environment, camera, style and constraints. Each design A→D locks more of them.
2. Lint the v1 prompt. It exits with status 1 and lists every problem: a missing constraints field, 'night' vs 'noon', 'close-up' vs 'wide shot', a repeated word and no references.

```
python3 prompt_lab.py compose --fields data/shot2-fields-v1.json --out out
```

3. Copy `data/shot2-fields-v2-template.json` to `out/shot2-fields-v2.json`. Fill `action`, `environment`, `camera`, `style` and `constraints`. Use one camera description only (for example: medium shot, eye level, 35mm, tracking left to right).

4. Put the constraints in plain words: 'keep coat and scarf colours exactly as the reference sheet; no extra people in the foreground; subject stays in the centre third'.

5. Lint your v2 until it exits cleanly (status 0). Read the prose and JSON versions it writes and check the `@element` references come first.

```
python3 prompt_lab.py compose --fields out/shot2-fields-v2.json --out out
```

6. Measure the 16 supplied takes. The script finds the raincoat by colour in every take and measures its hue, position, size and the background warmth.

```
python3 prompt_lab.py variance --takes reference/takes --out out
```

7. Read `out/variance-by-cell.csv`. The variance index should fall from A (1.0) to D. Record which single step (A→B, B→C, C→D) removed the most of each kind of drift.

8. Open `out/takes-grid.png` and confirm the numbers match what you see row by row.

9. Write `out/prompt-notes.md`: which fields you will always lock for The Last Ferry, and which you will leave free for variety (for example crowd and steam).

10. Optional extension: generate four takes of your v2 prompt on a fast, cheap draft model at 1K, save them as `cellE_01.png` ... into `reference/takes/` and rerun the variance command. Record the model ID and date.

Verify

v1 exits with status 1 and at least four problems; your v2 exits with status 0; the variance index falls in the order $A > B > C > D$.

Expected outputs

- `out/shot2-fields-v1.prompt.txt` and `.json` with the lint report
- `out/shot2-fields-v2.json`, `.prompt.txt` and `.prompt.json`
- `out/takes.csv`, `out/variance-by-cell.csv`, `out/takes-grid.png`
- `out/prompt-notes.md`

Evidence checklist

- Lint output for v1 and the clean lint output for your v2
- Your v2 prompt in prose and JSON with @element references
- Variance table for designs A-D with your reading of it
- Written list of always-locked and deliberately-free fields

Troubleshooting

Symptom	What to do
'no raincoat found'	The take has no saturated yellow region: check the file is one of the supplied takes or a real generation of MEI.
My v2 still reports a CONFLICT	Remove one of the two terms; describe the camera once.
Lint says REPEATED	Say it once. Repetition dilutes a prompt; it does not emphasise it.
My own generated takes vary more than cell A	Expected: the supplied takes are a designed simulation. Lock the reference images first.

Note: The same content, plus the full prompt pack, is in `labs/lab-03-structured-prompting-stills/README.md` and its PDF. Sources used in this lab: S7, S10, S11, S13, S15.

Lab 04 — State Variants, Clean-Up and Reframing with Masks

Outcome: ELO2 · **Abilities:** A2 · **Knowledge:** K4

Slide: 65 · **Folder:** `labs/lab-04-state-variants-reframing/` · **In class:** 15 minutes

Scenario

The HARBOUR PIER plate needs three edits before it becomes a reference: remove a stray traffic cone, make a second story state ('the ferry has pulled away — change nothing else'), and deliver a 9:16 version. A model returned the state edit; you verify it with pixel evidence before accepting it.

What you will produce

An inpainted plate with proof that nothing outside the mask moved, a verification heatmap and verdict for the state edit, and a 9:16 crop and outpaint mask with the percentage of pixels each approach keeps or must generate.

Tools: Python 3 · opencv-python · NumPy · optional: an image editor or generative edit tool

Files supplied with this lab

Path	What it is
reference/pier-docked.png	HARBOUR PIER state A with a stray orange cone (SIMULATED)
reference/pier-leaving-edit.png	State B as returned by a model: ferry leaving, plus one unrequested change (SIMULATED stand-in)
data/edit-jobs.json	Mask boxes, permitted edit zone, thresholds and targets
data/state-edit-prompt.md	The exact state-edit prompt that produced the edit
edit_lab.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Read `data/edit-jobs.json`. The mask convention here is WHITE = edit, BLACK = preserve. Note that some tools use the opposite — always check the contract.

2. Inpaint the cone. The script builds the mask from the box, dilates it by 4 px and runs two classical inpainting methods.

```
python3 edit_lab.py inpaint --image reference/pier-docked.png --jobs data/edit-jobs.json --out out
```

3. Confirm both methods leave 0 target (orange) pixels and a maximum change OUTSIDE the mask of 0. Open `out/inpaint_tela.png` and judge the planks by eye.

4. Read `data/state-edit-prompt.md`. The prompt asked for exactly one change: the ferry has left.

5. Verify the returned edit. Only the ferry zone (`allowed_box`) may change.

```
python3 edit_lab.py verify --before reference/pier-docked.png --after reference/pier-leaving-edit.png --jobs data/edit-jobs.json --out out
```

6. Read the verdict. The script finds an unrequested region at about [711, 48, 120, 120]. Open `out/edit-heatmap.png`: what changed there?

7. Decide: regenerate the state edit, or fix the lamp locally with a second masked edit? Write the decision and the prompt fix in `out/edit-decision.md`.

8. Reframe for vertical delivery.

```
python3 edit_lab.py reframe --image reference/pier-docked.png --target 9:16 --out out
```

9. Compare the two options: a centre crop keeps about 30% of the width and generates nothing; an outpaint keeps 100% of the source but must generate about 70% of the pixels (white in `out/outpaint_mask.png`).

10. Record which option you will use for the pier in the 9:16 cut-down, and why, in `out/edit-decision.md`.

11. Optional extension: send `outpaint_placeholder.png` and `outpaint_mask.png` to a generative-expand tool, save the result and run `verify` against the placeholder with an allowed box covering only the new areas.

Verify

Inpainting leaves 0 orange pixels and 0 change outside the mask; verify returns REGENERATE and names one unrequested region near the lamp; reframe reports both percentages.

Expected outputs

- `out/cone-mask.png`, `inpaint_telea.png`, `inpaint_ns.png`
- `out/edit-heatmap.png` and `out/edit-verify.json` with the verdict
- `out/reframe_crop.png`, `outpaint_mask.png`, `outpaint_placeholder.png`
- `out/edit-decision.md`

Evidence checklist

- Inpainting output showing zero change outside the mask
- Verification heat-map with the unrequested region marked, and the verdict
- Crop-versus-outpaint comparison with both percentages
- Written accept / regenerate / re-edit decision with the revised prompt

Troubleshooting

Symptom	What to do
'before/after sizes differ'	An edit must not resize the plate; re-export at the original size.
verify finds rain streaks as changes	Raise <code>pixel_threshold</code> slightly or rely on <code>min_region_px</code> ; rain is re-rendered noise.
Inpainting smears a large area	Classical inpainting diffuses neighbours; use a generative inpaint for large or textured regions.
My tool's mask is inverted	Invert with <code>cv2.bitwise_not(mask)</code> and state the convention in your notes.

Note: The same content, plus the full prompt pack, is in [labs/lab-04-state-variants-reframing/README.md](#) and its PDF. Sources used in this lab: S6, S13, S15, S17.

Lab 05 — Cinematic Grading and Enhancement Measured with PSNR and SSIM

Outcome: ELO2 · **Abilities:** A2 · **Knowledge:** K4

Slide: 66 · **Folder:** [labs/lab-05-grading-enhancement-metrics/](#) · **In class:** 15 minutes

Scenario

Raw generations arrive noisy, soft, flat or at draft resolution. Starting from the approved graded master of shot S2, you apply four typical defects, repair each with three candidate filters, and score every result with PSNR and SSIM. Then you apply the film's teal-and-orange look as a point operation.

What you will produce

An enhancement score table (4 defects × 3 repairs + unrepaired), a before/after sheet of the best repair per defect, and a flat-versus-graded comparison with the measured shift in shadow and highlight warmth.

Tools: Python 3 · opencv-python · NumPy · no network

Files supplied with this lab

Path	What it is
reference/graded-master.png	Approved graded master of shot S2 (SIMULATED)
data/degradations.json	The four defects, their parameters and the look strength
grade_lab.py	Runnable script
ssim.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Run the SSIM self-test. `cv2.quality` is contrib-only, so the course ships a NumPy SSIM.

```
python3 ssim.py
```

2. Read [data/degradations.json](#): sensor-like noise, soft focus, a flat log-like look and a half-resolution draft.

3. Run the lab. It degrades the master, repairs each defect three ways and scores everything against the master.

```
python3 grade_lab.py --master reference/graded-master.png --degradations data/degradations.json --out out
```

4. Open `out/enhancement-scores.csv`. For noise, note that PSNR prefers the bilateral filter while SSIM prefers the Gaussian blur. Explain why: PSNR averages per-pixel error; SSIM rewards preserved local structure.
5. For soft focus, compare unsharp amount 0.8 with 2.5. The stronger one scores higher PSNR but lower SSIM: look for halos on the raincoat edge in `out/before-after.png`.
6. For the flat look, compare the two level settings. Clipping too hard (60–190) destroys structure: SSIM falls below 0.4.
7. For the half-resolution draft, compare nearest, cubic and Lanczos + unsharp. This is why you generate drafts at low resolution and upscale only the keepers.
8. Open `out/graded-look.png`. The LUT pushes shadows towards teal and highlights towards orange. Record the printed warmth shift. A LUT changes each pixel on its own value: it cannot sharpen or denoise.
9. Write `out/grade-decisions.md`: the repair you choose per defect and the metric AND visual reason for each choice.
10. Optional extension: grade one of your own stills with the same LUT and compare its warmth numbers with the master's.

Verify

Every repair beats the unrepaired SSIM for its defect except two you should explain: nearest-neighbour upscaling (identical to the unrepaired draft) and the over-clipped levels setting; the look shifts shadows cooler and highlights warmer.

Expected outputs

- `out/enhancement-scores.csv`
- `out/before-after.png`
- `out/graded-look.png`
- `out/grade-decisions.md`

Evidence checklist

- Score table with PSNR and SSIM for every repair
- One case where PSNR and SSIM disagree, explained by mechanism
- Flat-versus-graded image with the warmth measurements
- Chosen repair per defect with metric and visual justification

Troubleshooting

Symptom	What to do
ImportError: ssim	Run from inside the lab folder so <code>ssim.py</code> is importable.

Symptom	What to do
fastNIMeans is slow	It is the most expensive filter; it still runs in a few seconds at 960x540.
PSNR is inf	The two images are identical — check you compared the repair, not the master.
My grade looks garish	Lower look_strength in degradations.json to 0.15 and rerun.

Note: The same content, plus the full prompt pack, is in [labs/lab-05-grading-enhancement-metrics/README.md](#) and its PDF. Sources used in this lab: S9, S11, S14, S17.

Topic 03 — AI Visual Features, Character Consistency and Scene Design

LU3 · ELO3 · knowledge K5, K6, K7 · abilities A3, A4

Slides 67–85 · 75 minutes classroom facilitation · 60 minutes practical

Features as the language of consistency · character reference sheets and AI casting · local features — colour, edge, texture and keypoints · global features — histograms, statistics and moments for the look of a shot · shot size, camera angle, movement and lighting

Key concepts

Consistency is a feature-matching problem. A character is 'the same' when repeatable features match across shots: wardrobe colours, silhouette, facial landmarks, a distinctive prop. The video model conditions on those features, and your QA measures them.

A character reference sheet is a feature specification. Front, side and back views plus close-ups and expressions, one outfit, fixed colours. Casting attributes — genre, era, archetype, identity, build, details, outfit — pin the look before the first shot is generated.

Local features answer 'what is here?' Dominant colour in HSV, Canny edges of the silhouette, texture energy and ORB keypoints on the face or prop. They are compared crop to crop, so they catch a scarf that changed colour or a prop that lost its clasp.

Global features answer 'does this whole shot belong?' HSV histograms compared with Bhattacharyya distance, mean luminance, contrast, warm/cool balance and Hu moments of the composition mask. Together they form a numeric 'look bible' that every shot must stay close to.

Scene design is composition, camera and light. Shot size (wide, medium, close-up), angle (eye-level, low, high, over-the-shoulder), movement (push-in, pan, tracking, handheld) and lighting (key, fill, rim, practicals, blue hour). The rule of thirds and the 180-degree rule keep a cut readable.

Locations and props need angles and states. Generate a multi-angle location grid and each story state of a location (ferry docked, ferry leaving) as its own reference, plus a clean hero plate of every prop. The model reuses what you give it and invents what you don't.

Lab 06 — Character Reference Sheets and Consistency Features

Outcome: ELO3 · **Abilities:** A4 · **Knowledge:** K5, K6

Slide: 84 · **Folder:** [labs/lab-06-character-consistency-features/](#) · **In class:** 30 minutes

Scenario

MEI appears in six generated shots. Three of them have drifted from her reference sheet in ways a tired reviewer can miss. You design a consistency check from local features — colour, edge, texture and ORB keypoints — implement it on the character and prop crops, set the thresholds, and prove which shots must be regenerated.

What you will produce

A feature-extraction design (which feature detects which failure), an identity report for six shots with pass/drift verdicts and reasons, and an identity strip comparing each shot crop with the reference sheet.

Tools: Python 3 · opencv-python · NumPy · optional: an image model with character references

Files supplied with this lab

Path	What it is
reference/mei-reference-sheet.png	MEI reference sheet: front, side, back, close-up and the RED BOX (SIMULATED)
reference/shots/shot_01.png ... shot_06.png	Six shots of MEI, three with injected drift (SIMULATED)
data/crops.json	Character, prop and clasp boxes per shot (generated with the frames)
data/identity-thresholds.json	Colour ranges and starting thresholds
character_features.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Open the reference sheet. List every feature a viewer uses to recognise MEI: yellow hooded raincoat, red scarf, black bag with a chequered patch, the RED BOX with a brass clasp. This list is your feature specification.

2. Complete a design table in `out/feature-design.md` with three columns: failure mode (coat colour drift, scarf change, lost prop detail, different person), the feature that detects it, and why that feature is repeatable across shots.

3. Read `data/crops.json`. Each shot has a character box, a prop box and a clasp box. In production these boxes come from the Lab 08 detector or a manual mark-up.

4. Extract the coat colour of the sheet crop in HSV. OpenCV 8-bit hue runs 0–179, so yellow sits near 22–23.

```
python3 -c "import cv2,json,numpy as np;s=cv2.imread('reference/mei-reference-sheet.png');x,y,w,h=json.load(open('data/crops.json'))['crops']['sheet']['character'];hsv=cv2.cvtColor(s[y:y+h,x:x+w],cv2.COLOR_BGR2HSV);m=cv2.inRange(hsv,(5,150,170),(40,255,255))>0;print(np.median(hsv[...][m]))"
```

5. Run the full check. It extracts colour, k-means dominant colours, Canny edge density, Laplacian texture energy and ORB keypoint matches for every shot.

```
python3 character_features.py --sheet reference/mei-reference-sheet.png --shots reference/shots --crops data/crops.json --thresholds data/identity-thresholds.json --out out
```

6. Read the verdicts. Expect DRIFT on shot_04 (scarf), shot_05 (clasp missing) and shot_06 (coat hue 13 instead of 23). Match each to the feature that caught it.

7. Look at `orb_good_matches`: shot_06 still matches the sheet with about 12 good matches. Explain why a keypoint match on the bag patch cannot see a colour change.

8. Look at the brass measurement. The raincoat and the brass clasp share a hue, which is why the check measures brass only inside the small clasp window. Record this as a design decision.

9. Tighten `coat_hue_tolerance` from 4 to 1 and rerun. Does any good shot now fail? Restore the value and write down the tolerance you would ship and why.

10. Open `out/identity-strip.png` (sheet first, red = drift) and check the verdicts by eye.

11. Write the regeneration list in `out/regenerate.md`: shot, failed feature, and the prompt or reference fix (for example: re-attach the sheet; add 'red scarf exactly as reference' to constraints).

12. Optional extension: save your own generated shots of a character with a reference sheet, add their boxes to `crops.json` and run the same check.

Verify

shot_01, shot_02 and shot_03 pass; shot_04, shot_05 and shot_06 each fail on exactly the feature designed to catch them.

Expected outputs

- out/feature-design.md
- out/identity-report.csv
- out/identity-strip.png
- out/regenerate.md

Evidence checklist

- Feature-design table mapping each failure mode to a feature
- Identity report with verdicts and reasons for all six shots
- Threshold experiment with the tolerance you would ship
- Regeneration list with a prompt or reference fix per failed shot

Troubleshooting

Symptom	What to do
coat hue is nan	No pixel fell in the coat HSV range: check the crop box and the ranges in identity-thresholds.json.
ORB finds 0 matches	Crops must contain texture; flat colour has no corners. Include the bag patch or the face.
A good shot fails on brass	Check the clasp box is on the clasp; lower min_clasp_brass_cover and justify it.
k-means results change slightly between runs	k-means uses random seeds; the dominant colour, not its exact value, is the feature.

Note: The same content, plus the full prompt pack, is in [labs/lab-06-character-consistency-features/README.md](#) and its PDF. Sources used in this lab: S6, S7, S9, S17.

Lab 07 — Look Bible and Scene Continuity with Global Descriptors

Outcome: ELO3 · **Abilities:** A3 · **Knowledge:** K7

Slide: 85 · **Folder:** [labs/lab-07-look-bible-scene-continuity/](#) · **In class:** 30 minutes

Scenario

Eight shots of The Last Ferry have been generated across the NIGHT MARKET and the HARBOUR PIER. Two of them do not belong to the film's look. You analyse global descriptors — histograms, brightness, contrast, saturation, warmth, tint, Hu moments and edge density — against the approved master of each location, and write the film's numeric look bible.

What you will produce

A look report for eight shots with the descriptor that moved for every off-look shot, a colour-coded look strip, and a look bible with the limits you would ship.

Tools: Python 3 · opencv-python · NumPy · no network

Files supplied with this lab

Path	What it is
reference/look-master-market.png	Approved look master, NIGHT MARKET (SIMULATED)
reference/look-master-pier.png	Approved look master, HARBOUR PIER (SIMULATED)
reference/sequence/shot_01.png ... shot_08.png	Eight generated shots, two off-look (SIMULATED)
data/look-bible.json	The look in words, the masters per location and the limits
data/sequence-manifest.csv	Which location each shot belongs to
look_audit.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Read the `look` sentence in `data/look-bible.json`. Underline the parts a number could check: blue hour (brightness), teal shadows and warm practicals (warmth, tint), restrained saturation.

2. Compute one global descriptor by hand: the 2-D hue/saturation histogram of the pier master, normalised to sum to 1.

```
python3 -c "import cv2;h=cv2.cvtColor(cv2.imread('reference/look-master-pier.png'),cv2.COLOR_BGR2HSV);H=cv2.calcHist([h],[0,1],None,[30,32],[0,180,0,256]);print(H.shape, float(H.sum()))"
```

3. First, try the WRONG design: temporarily set both entries of `masters` to the pier master and run the audit. Every market shot is flagged — a single master punishes legitimate differences between locations. Restore the file.

```
python3 look_audit.py --sequence reference/sequence --manifest data/sequence-manifest.csv --bible data/look-bible.json --out out
```

4. Run the audit with one master per location (the restored bible).

```
python3 look_audit.py --sequence reference/sequence --manifest data/sequence-manifest.csv --bible data/look-bible.json --out out
```

5. Read the verdicts. `shot_05` is OFF LOOK on exposure, white balance and saturation — a 'noon' generation. `shot_07` is OFF LOOK on tint and saturation — a magenta cast.

6. Explain the geometrical descriptor: Hu moments of the bright-practicals mask describe where the light sits in the frame. Which shots have the largest `hu_dist`, and is that a look problem or a composition choice?

7. Read the `jump_from_prev` column. Large jumps at location changes are expected; a large jump between two shots of the SAME location is a continuity warning.

8. Open `out/look-strip.png` (red = off look) and check it against your eye.

9. Tune one limit (for example `max_abs_d_mean_v`) so the six good shots pass with a margin and both bad shots still fail. Record the value and the margin.

10. Write `out/look-bible.md`: the look in one sentence, the per-location masters, the limits you ship, and the regrade or regenerate action for `shot_05` and `shot_07`.

11. Optional extension: grade `shot_05` with the Lab 05 LUT and levels and re-audit it. Can a grade rescue a daylight generation, or must it be regenerated?

Verify

Six shots are ON LOOK and exactly `shot_05` and `shot_07` are OFF LOOK, each with the descriptors that moved.

Expected outputs

- `out/look-report.csv`
- `out/look-strip.png`
- `out/look-bible.md`

Evidence checklist

- Evidence of the single-master failure and the per-location fix
- Look report with the moved descriptors named for `shot_05` and `shot_07`
- Tuned limit with the measured margin
- Look bible with limits and a regrade-or-regenerate decision per off-look shot

Troubleshooting

Symptom	What to do
'shots missing from the manifest'	Add a row for every shot file in <code>data/sequence-manifest.csv</code> .
Every shot is OFF LOOK	Check each shot is compared with the master of its own location.
<code>hist_dist</code> is high for a good shot	Framing changes the histogram; rely on exposure, warmth and tint as the tight gates.
cannot read master	Paths in <code>look-bible.json</code> are relative to the lab folder; run from there.

Note: The same content, plus the full prompt pack, is in `labs/lab-07-look-bible-scene-continuity/README.md` and its PDF. Sources used in this lab: S8, S9, S14, S17.

Topic 04 — AI Video Generation, Animation and Video Analytics

LU4 + LU5 · ELO4 + ELO5 · knowledge K8, K9, K10 · abilities A5, A6

Slides 86–113 · 150 minutes classroom facilitation · 180 minutes practical

Deep learning for video — temporal coherence, start/end frames and video references · multi-shot and timeline prompting with dialogue and emotion · detecting and verifying the

cast · voiceover-first sound design, captions and export · shot, motion, tracking and drift analytics

Key concepts

Deep video models learn motion from data. Convolution and attention layers learn spatial structure; temporal attention links frames. Each frame must look right (spatial coherence) AND agree with its neighbours (temporal coherence): flicker, morphing and identity drift are temporal failures.

Continuity is conditioned, shot to shot. A start frame fixes where a shot begins; an end frame fixes where it lands. A video reference or extension passes the whole previous clip, so lighting, mood and motion carry across the cut. Clips are typically capped at 5-15 s.

Write shots, not paragraphs. Multi-shot prompts list Shot 1, Shot 2, Shot 3 in order; timeline prompts give second-by-second beats. Dialogue goes in quotation marks, and each character gets an emotion and an intensity. Dense action: generate three takes and pick.

Detection and segmentation turn frames into evidence. A face or person detector counts the cast in every shot; a segmentation mask isolates the subject for grading. Evaluate detectors with IoU, precision and recall before you trust them to approve a shot.

Video analytics finds cuts, motion, drift and events. Histogram differences find shot boundaries; optical flow classifies camera movement; CAMShift tracks the courier's raincoat; hue drift and luminance flicker become numbers; the first frame the subject appears is an event.

Sound drives picture. Generate the voiceover first and cut the storyboard to its timing. Direct delivery with audio tags, score music by mood and tempo, add sound effects, then caption inside the safe area and export one master per platform.

Lab 08 — Cast-Presence Detection and Detector Evaluation

Outcome: ELO4 · **Abilities:** A5 · **Knowledge:** K8, K9

Slide: 101 · **Folder:** [labs/lab-08-cast-detection-evaluation/](#) · **In class:** 60 minutes

Scenario

A generated shot can quietly drop a character or add an extra. Before a detector is allowed to approve shots automatically, it must be evaluated. You run two built-in machine-learning detectors (a Viola-Jones face cascade and a HOG + linear-SVM people detector) on twelve storyboard frames with exact truth, implement IoU matching, sweep their operating parameters, and then use the face detector to check each frame's cast against the shot list.

What you will produce

An evaluation table (TP, FP, FN, precision, recall, F1 at three IoU thresholds), a parameter sweep with a chosen operating point, a cast-check table separating CAST DRIFT from DETECTOR ERROR, and a positive-control run on an AI-generated still.

Tools: Python 3 · opencv-python · NumPy · no weights download, no network

Files supplied with this lab

Path	What it is
reference/frames/frame_00.png ... frame_11.png	Twelve schematic storyboard frames with exact truth (SIMULATED)
reference/positive-control/retail-adults-reference.png	AI-GENERATED still of two fictional adults, with PROVENANCE.md
data/eval-config.json	IoU thresholds, parameter sweeps, matching rule and the operating-point rule
detect_eval.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Read `data/eval-config.json`: the three IoU thresholds, the minNeighbors and SVM weight sweeps, the one-to-one matching rule and the operating-point rule.

2. Confirm both detectors ship inside opencv-python — no download.

```
python3 -c "import cv2,os;p=cv2.data.haarcascades+'haarcascade_frontalface_default.xml';print(os.path.exists(p), len(cv2.HOGDescriptor_getDefaultPeopleDetector()))"
```

3. Compute IoU by hand for [0, 0, 100, 100] and [50, 0, 100, 100]: intersection 5000, union 15000, IoU = 1/3. The script self-tests the same case before it runs.

4. Run the evaluation with the cast check.

```
python3 detect_eval.py --frames reference/frames --truth data/ground-truth.json --config data/eval-config.json --cast data/shot-cast.json --out out
```

5. Read the evaluation lines. The HOG person detector scores well at IoU 0.3 and 0.5 and collapses at 0.7: its boxes are the right objects with loose edges. Explain why the IoU threshold is part of the metric, not a detail.

6. Open `out/pr-sweep.csv`. Apply the operating-point rule: maximise recall subject to precision ≥ 0.60 . Record the minNeighbors and SVM weight you choose.

7. Read the CAST CHECK. `frame_04` and `frame_09` disagree with the shot list and with the detector's truth count: CAST DRIFT — the frame is wrong. `frame_06` disagrees only because the detector fired twice: DETECTOR ERROR.

8. Open `out/overlay_frame_00.png` (green = truth, blue = face, red = person) and check one frame by eye.

9. Run the positive control on the AI-generated still.

```
python3 detect_eval.py --frames reference/positive-control --truth
data/positive-control-truth.json --config data/eval-config.json --out
out/positive-control
```

10. Compare: the detectors that were near-perfect on schematic frames produce many false positives on a realistic image. Write two sentences on domain shift and why one image proves nothing either way.

11. Decide in `out/detector-decision.md`: may the face detector auto-approve shots, or only route disagreements to a reviewer? Use your numbers.

12. Explain in one paragraph how a segmentation mask would differ from a detection box for isolating MEI to regrade her raincoat.

Verify

The IoU self-test passes; the evaluation prints nine detector rows; the cast check flags `frame_04` and `frame_09` as CAST DRIFT and `frame_06` as DETECTOR ERROR.

Expected outputs

- `out/detections.json`, `out/evaluation.csv`, `out/pr-sweep.csv`
- `out/cast-check.csv` and `out/overlay_frame_00.png`
- `out/positive-control/evaluation.csv`
- `out/detector-decision.md`

Evidence checklist

- Hand IoU calculation matching the self-test
- Evaluation table at three IoU thresholds with your explanation of the 0.7 collapse
- Chosen operating point with the precision constraint shown
- Cast-check table distinguishing CAST DRIFT from DETECTOR ERROR
- Positive-control result and the auto-approve decision

Troubleshooting

Symptom	What to do
'cascade failed to load'	Reinstall opencv-python; cv2.data.harcascades points inside the package.
'frame files and truth frame keys must match'	Every image in --frames needs an entry in the truth file, even an empty list.
The positive-control run shows many false positives	Expected: that is the domain-shift lesson. Do not tune on one image.
Different numbers on another machine	Detector output can vary slightly by OpenCV version; record yours.

Note: The same content, plus the full prompt pack, is in `labs/lab-08-cast-detection-evaluation/README.md` and its PDF. Sources used in this lab: S13, S14, S17.

Lab 09 — Storyboard to Shot Prompts, Start and End Frames, and the Animatic

Outcome: ELO4 · **Abilities:** A5, A6 · **Knowledge:** K8, K10

Slide: 102 · **Folder:** [labs/lab-09-storyboard-shot-prompts/](#) · **In class:** 30 minutes

Scenario

The Last Ferry is planned as six shots and six voiceover lines. Before generating a single clip, you validate the shot list against the voiceover and the model's clip cap, fix the two problems the check finds, write every shot as a six-field prose prompt, a multi-shot timeline prompt with dialogue and emotions, and JSON, and render a timed animatic from the storyboard's start and end frames.

What you will produce

A corrected shot list that passes every check, a shot-prompt pack (prose, timeline and JSON per shot, with start/end frames and takes), and a 48-second animatic with the voiceover lines burned in.

Tools: Python 3 · opencv-python · NumPy · optional: any image-to-video model with start/end frames

Files supplied with this lab

Path	What it is
reference/board/shot1_start.png ... shot6_end.png	Storyboard start and end keyframes for six shots (SIMULATED)
data/film-brief.md	One-page brief: cast, locations, prop, beats and sound
data/shot-list.json	Six shots with size, angle, movement, lens, beats, elements and takes
data/vo-timing.json	Six voiceover lines with timing, emotion and intensity
storyboard.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Read `data/film-brief.md` and `data/shot-list.json`. Every shot references saved elements (`@mei`, `@red_box`, ...) and repeats one identical continuity block.
2. Read `data/vo-timing.json`. The voiceover was generated FIRST; the storyboard is cut to it, not the other way round.
3. Validate. The check exits with status 1 and reports two problems.

```
python3 storyboard.py check --shots data/shot-list.json --vo data/vo-timing.json
```

4. Problem 1: S6 is 12 s but the model caps clips at 10 s. Copy the shot list to `out/shot-list-v2.json`, set S6 to 6 s, and add S7 (6 s) with `"extends": "S6"` — a video extension that starts on S6's last frame.

5. Problem 2: the line 'No sender. Just a tag, folded shut.' ends at 22.5 s but S3 cuts at 22.0 s. Copy the VO file to `out/vo-timing-v2.json` and end the line at 21.8 s (or move the cut — record which and why).

6. Re-run the check on your v2 files until it passes.

```
python3 storyboard.py check --shots out/shot-list-v2.json --vo out/vo-timing-v2.json
```

7. Write the prompt pack.

```
python3 storyboard.py prompts --shots out/shot-list-v2.json --vo out/vo-timing-v2.json --out out
```

8. Read `out/shot-prompts.md`. In each timeline prompt the beats and dialogue are ordered by time, dialogue is in quotation marks and each line carries an emotion and intensity. S2 asks for three takes because it is dense action.

9. Render the animatic.

```
python3 storyboard.py animatic --shots out/shot-list-v2.json --vo out/vo-timing-v2.json --board reference/board --out out
```

10. Watch `out/animatic.mp4` (48.00 s). Is any line crowded or any shot too long to hold interest? Note one timing change you would make.

11. Write `out/continuity-plan.md`: which shots use a start frame, which use start AND end frames, and which use a video reference/extension, and why each choice keeps lighting and mood across the cut.

12. Optional extension: generate S1 on an image-to-video model with its start and end frames and the timeline prompt. Record model, duration, resolution and takes.

Verify

Your v2 files pass every check; the prompt pack has one entry per shot; the animatic is exactly 48.00 s.

Expected outputs

- `out/shot-list-v2.json` and `out/vo-timing-v2.json` passing the check
- `out/shot-prompts.md` and `out/shot-prompts.json`
- `out/animatic.mp4` (48.00 s)
- `out/continuity-plan.md`

Evidence checklist

- The failing check output and your two fixes

- The passing check output for the v2 files
- One shot's prose, timeline and JSON prompts, with start/end frames and takes
- The animatic and one timing change you would make
- Continuity plan: start frame, start/end frames or video extension per shot

Troubleshooting

Symptom	What to do
'missing keyframes for S7'	Add "extends": "S6" to S7 so it starts on S6's last frame.
'references @x, which is not a saved element'	Add the element to the film's elements list or fix the spelling.
'continuity block differs'	Copy the film's continuity_block into the shot exactly — any edit invites drift.
The animatic will not play	Open it in VLC; mp4v is widely supported but not by every browser.

Note: The same content, plus the full prompt pack, is in [labs/lab-09-storyboard-shot-prompts/README.md](#) and its PDF. Sources used in this lab: S6, S7, S11, S16.

Lab 10 — Shot, Motion and Continuity Analytics on the Assembled Sequence

Outcome: ELO5 · **Abilities:** A6 · **Knowledge:** K10

Slide: 112 · **Folder:** [labs/lab-10-shot-motion-continuity-analytics/](#) · **In class:** 60 minutes

Scenario

Five generated shots have been assembled into a 16-second sequence. Somewhere in it there is a flickering shot and a character whose coat slowly changes colour. You build the analytics that find them automatically: cut detection, background camera motion, a flicker index, CAMShift tracking of MEI's raincoat, hue-drift measurement and the first-appearance event — and score each against exact truth.

What you will produce

A shot report (boundaries, camera movement, flicker), a tracking report (IoU, centre error, jitter, hue drift), the first-appearance event, and a timeline graphic, each compared with the truth file.

Tools: Python 3 · opencv-python · NumPy · no network

Files supplied with this lab

Path	What it is
reference/generated-sequence.mp4	16-second, five-shot sequence with injected flicker and identity drift (SIMULATED clip)
data/analytics-config.json	Cut, motion, flicker, colour and drift parameters

Path	What it is
data/sequence-truth.json	Exact cuts, defects, subject boxes and first appearance (generated with the clip)
continuity.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Read `data/sequence-truth.json`: 384 frames at 24 fps, cuts at frames 72, 168, 216 and 312, a flicker defect in one shot and an identity-drift defect in another.

2. Read `data/analytics-config.json`. A cut needs TWO signals: a pixel-difference jump above its recent level AND a histogram change. Write down why one signal is not enough.

3. Run the analytics.

```
python3 continuity.py --clip reference/generated-sequence.mp4 --truth
data/sequence-truth.json --config data/analytics-config.json --out out
```

4. Check the cuts: 4/4 found, 0 false. Now set `cut_min_hist` to 0 and rerun: the flicker shot produces false cuts. Restore 0.08 and record the evidence.

5. Check camera motion. S1 is a pan right (background moves about -5 px per frame). The estimate uses corner features on the BACKGROUND only, with the subject box excluded and a RANSAC fit. Explain why including the subject would label S2 a pan.

6. Check flicker: exactly one shot (S3, the box close-up) exceeds the flicker threshold. Give the index value and the threshold.

7. Check the event: MEI first appears at frame 72 (3.00 s). This is the timecode you would put in the edit decision list for her entrance.

8. Check tracking in S2: IoU(x) about 0.8 and jitter about 1 px — a stable track, and the coat hue stays at 23.

9. Check S4: the coat hue drifts from 23 to 13 and the colour tracker collapses (large jitter). Explain why identity drift both is detected by the hue measure AND breaks a colour-based tracker.

10. Open `out/timeline.png`: the red band marks the flicker shot, the orange band the drift shot, and the spikes are cuts.

11. Write `out/continuity-notes.md` for the editor: each defect, its timecode, the measure that found it, and the fix (regenerate S4 with the reference sheet re-attached; deflicker or regenerate S3).

12. Optional extension: export your own generated clips as one file with a matching truth file of cut frames and run the same analytics.

Verify

Cuts 4/4 with 0 false; S1 labelled pan right; only S3 flagged FLICKER; first appearance at frame 72; S2 stable and S4 DRIFT.

Expected outputs

- out/shot-report.csv
- out/tracking-report.csv
- out/timeline.png
- out/continuity-report.json
- out/continuity-notes.md

Evidence checklist

- Cut results against truth, and the false-cut evidence with cut_min_hist = 0
- Camera-motion result per shot with the background-only explanation
- Flicker index per shot with the threshold
- Tracking and hue-drift results for S2 and S4
- Editor's continuity notes with timecodes and fixes

Troubleshooting

Symptom	What to do
No cuts detected	Check cut_min_diff and cut_ratio; print the per-frame pixel difference to see the spikes.
Camera motion reads 'static' for everything	Too few background features: lower min_bg_features or goodFeaturesToTrack quality.
First appearance at frame 0	Lanterns or windows matched the colour: use the tight coat range and the largest-blob rule.
CamShift box explodes in S4	Expected: the coat left the tracked hue. That is the drift evidence.

Note: The same content, plus the full prompt pack, is in [labs/lab-10-shot-motion-continuity-analytics/README.md](#) and its PDF. Sources used in this lab: S6, S13, S14, S17.

Lab 11 — Voiceover, Music, Captions and the Final Export Ladder

Outcome: ELO4 · **Abilities:** A5, A6 · **Knowledge:** K9, K10

Slide: 113 · **Folder:** [labs/lab-11-sound-captions-final-export/](#) · **In class:** 30 minutes

Scenario

The last ten seconds of The Last Ferry are picture-locked. You finish them: validate the subtitles against the house caption rules, build and measure a placeholder mix (voiceover cues, a music bed at the brief's tempo, rain and a ferry horn, with the music

ducked under the voice), and export a 16:9 master, a 9:16 Reel and a 4:5 feed version that follow MEI using her raincoat's segmentation mask.

What you will produce

A caption file that passes every rule, an audio report (peak, RMS, voice-over-music margin), and three exported masters with burned-in captions — muxed to H.264/AAC when FFmpeg is installed.

Tools: Python 3 · opencv-python · NumPy · optional FFmpeg · optional: a voice, music and editing tool

Files supplied with this lab

Path	What it is
reference/picture-lock.mp4	10-second picture lock of the last three shots (SIMULATED clip)
data/captions.srt	Subtitle file with five deliberate faults
data/caption-rules.json	Line count, line length, reading speed, duration and gap rules
data/vo-cues.json	Voiceover cue sheet with audio tags
data/music-brief.json	Music prompt, tempo, ducking and level targets
data/export-profiles.json	16:9, 9:16 and 4:5 profiles with safe areas
finish.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Read `data/vo-cues.json`. Each line carries an audio tag such as [out of breath] or [stunned whisper] — that is how delivery is directed line by line.

2. Read `data/music-brief.json`: the music prompt, 86 BPM, the ducking depth, the peak ceiling and the minimum voice-over-music margin.

3. Validate the captions. The check exits with status 1 and lists five faults.

```
python3 finish.py captions --srt data/captions.srt --rules data/caption-rules.json
```

4. Copy the file to `out/captions-v2.srt` and fix it: delete the stray 'Four minutes.' cue, shorten cue 3 to 'No sender. / Just a tag, folded shut.' on two lines, and renumber. Re-run the check on your file until it passes.

```
python3 finish.py captions --srt out/captions-v2.srt --rules data/caption-rules.json
```

5. Build and measure the placeholder mix.

```
python3 finish.py audio --srt out/captions-v2.srt --out out
```

6. Read the audio report: peak below -1 dBFS, and voice at least 6 dB above the music while the voice is active. Change `duck_db` to 0 and rerun: record the new margin, then restore 9.

7. Export everything.

```
python3 finish.py all --srt out/captions-v2.srt --out out
```

8. Read the export ladder: the 9:16 Reel keeps about 31.6% of the master and follows MEI's raincoat mask; the 4:5 keeps about 45%. With FFmpeg each file is muxed to H.264 video and AAC audio.

9. Open `out/reels_9x16.mp4` (or `_final.mp4`). Check that the captions sit above the platform UI zone and that the SIMULATED label survived the crop.

10. Write `out/delivery-checklist.md`: captions pass, peak and margin, sizes and frame counts, subject in frame, provenance label present, AI-content disclosure set on upload.

11. Optional extension: replace the placeholder with a real voiceover and music track generated from the cue sheet and brief, then re-measure.

Verify

Your caption file passes every rule; the mix peaks at or below -1 dBFS with a voice margin of at least 6 dB; three exports are written with 240 frames each.

Expected outputs

- `out/captions-v2.srt` passing the check
- `out/mix-placeholder.wav` and `out/audio-report.json`
- `out/master_16x9.mp4`, `reels_9x16.mp4`, `feed_4x5.mp4` (+ `_final.mp4` with FFmpeg)
- `out/export-report.json` and `out/delivery-checklist.md`

Evidence checklist

- Failing caption check and your corrected caption file passing
- Audio report, and the margin with ducking switched off
- Export report with sizes, frame counts and kept percentages
- Delivery checklist including provenance and AI-content disclosure

Troubleshooting

Symptom	What to do
'Fix the caption problems before exporting'	Burned-in captions cannot be edited; fix the SRT first.
'ffmpeg not found: video only'	Install FFmpeg (brew install ffmpeg / winget install ffmpeg) or mux in your editor.
Captions overlap the platform buttons	Raise <code>safe.bottom</code> for that profile in <code>export-profiles.json</code> .
The vertical crop loses MEI	<code>follow_subject</code> must be true; check the coat colour range finds her.

Note: The same content, plus the full prompt pack, is in [labs/lab-11-sound-captions-final-export/README.md](#) and its PDF. Sources used in this lab: S6, S7, S8, S10, S11.

Topic 05 — Cloud and Edge AI Workflows for Short Film Production

LU6 · ELO6 · knowledge K11, K12, K13 · abilities A7, A8

Slides 114–128 · 75 minutes classroom facilitation · 60 minutes practical

The four-layer production architecture · all-in-one agents, multi-tool cloud pipelines and local open models · the protocols that move prompts, references and renders · cost, latency, privacy, likeness and disclosure constraints · measured cloud-versus-local trade-offs

Key concepts

A production system has four layers. Assets (script, references, voice, footage) -> local/edge workstation (pre-processing, QA, edit, grade, optional local models) -> cloud generation (image, video, voice and music services) -> delivery (platforms, archive).

Three workflow patterns, three control levels. An all-in-one agent turns one prompt and one photo into a finished drama. A multi-tool cloud pipeline chains planner, image, video, voice, music and editor. A local or hybrid pipeline runs open models on your own GPU.

Protocols are chosen by payload and wait time. HTTPS + JSON to submit a job; a long-running operation you poll, or a webhook that calls you back; WebSocket progress events from a local node server; signed-URL uploads to object storage for references and renders; HLS for streaming delivery.

Cost and time scale with shots x takes x seconds x resolution. Cost per finished minute = total credits spent / minutes that survive the edit. Explore at low resolution, generate extra takes only for dense action, and upscale only the keepers.

Constraints are gates, not weights. Consent for any real face or voice, licence terms for commercial use, personal-data rules (PDPA), platform AI-content disclosure, provenance (C2PA, SynthID) and brand safety eliminate options before cost is argued.

Local compute is bounded by memory. Weight memory = parameters x bytes per parameter; quantising from 32-bit to b-bit scales it by b/32. Local keeps data private and has no per-clip fee; cloud is faster to start, higher quality and needs no GPU.

Lab 12 — Cloud, Local and Hybrid Production Architecture Evaluation

Outcome: ELO6 · **Abilities:** A7, A8 · **Knowledge:** K11, K12, K13

Slide: 128 · **Folder:** [labs/lab-12-cloud-local-production-architecture/](#) · **In class:** 60 minutes

Scenario

Harbourline wants The Last Ferry and, if it works, a twice-weekly series. You design the production system: an all-in-one drama agent, a multi-tool cloud pipeline, a hybrid (cloud generation, local post-production) or a local open video model. You measure the local stages on your own laptop, model generation cost and time for each architecture, check whether the open model fits your GPU, apply the gates, and defend a recommendation.

What you will produce

A four-layer architecture diagram for your chosen design, an architecture matrix (cost, cost per finished minute, generation time, upload time, end-to-end hours, VRAM fit, gates), the protocol sequence for your design, and two what-if results.

Tools: Python 3 · opencv-python · NumPy · a pen or diagram tool · no paid service required

Files supplied with this lab

Path	What it is
reference/bench-clip.mp4	4-second 960x540 clip used to time the local stages (SIMULATED clip)
reference/element-pack/	The saved references (character sheet, two location plates) that a cloud architecture must upload
data/production-plan.json	Seven shots, seconds and takes for The Last Ferry
data/architectures.json	Four architectures: ILLUSTRATIVE rates, speeds, regions, quality, control and protocols
data/constraints.json	Budget, deadline, uplink, regions, likeness, quality, control and GPU memory
production_arch.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Draw the four layers for The Last Ferry in `out/architecture.png` or on paper: ASSETS (script, element pack, voice) → LOCAL WORKSTATION (QA analytics, grade, captions, mix, export) → CLOUD GENERATION (image, video, voice, music services) → DELIVERY (platforms, archive). Mark what crosses each boundary.
2. Read `data/architectures.json`. Note that every rate is ILLUSTRATIVE and must be replaced with the current price list before quoting a client.
3. Read `data/constraints.json`: S\$400 budget, 24-hour deadline, 20 Mbps uplink, a brand-hero quality tier of 4 and shot-level control required.
4. Run the evaluation.

```
python3 production_arch.py --plan data/production-plan.json --archs
data/architectures.json --constraints data/constraints.json --clip
reference/bench-clip.mp4 --elements reference/element-pack --out out
```

5. Section 1 is MEASURED on your machine: seconds of work per second of footage for decode, QA analytics, grading and encoding. Compare with a classmate's laptop.
6. Section 2 is MODELLED: 98 generated seconds for a 48-second film because of extra takes. Check one architecture's cost by hand: generated seconds × credits per second × S\$ per credit + fixed cost.
7. Read the VRAM line for the local model: a 14-billion-parameter model needs about 36 GB at FP16, 18 GB at INT8 and 9 GB at INT4 (weights × bytes × 1.3 for activations). Only INT4 fits a 16 GB GPU.
8. Read the gates. A fails quality and shot control; D fails quality. B and C pass; C is cheaper. Write why a gate is applied before ranking.
9. Describe the protocol sequence of your recommended architecture in `out/protocol-sequence.md`: signed-URL upload of the element pack, JSON POST per shot, the operation ID, polling or webhook, and the signed-URL download.
10. What-if 1: set `likeness_data_must_stay_local` to true (as it would be for a film using a real employee's face) and rerun. Record which architectures survive.
11. What-if 2: restore it, then lower `min_quality_tier` to 2 (a social draft). Rerun and record how the recommendation and cost per finished minute change.
12. Write `out/recommendation.md`: the architecture, the numbers that justify it, the gate that would overturn it, and the disclosure and provenance steps (C2PA / watermark, platform AI label) your pipeline performs before delivery.

Verify

The matrix lists four architectures; A and D fail gates with named reasons; C is recommended; both what-ifs are recorded with their new outcome.

Expected outputs

- `out/architecture-matrix.csv` and `out/architecture-report.json`
- `out/bench-encode.mp4` (the timed local encode)
- `out/architecture.png` (or a photo of your drawing)
- `out/protocol-sequence.md` and `out/recommendation.md`

Evidence checklist

- Four-layer architecture diagram with the boundary payloads marked
- MEASURED local timings from your machine
- Hand check of one MODELLED cost and the VRAM calculation
- Protocol sequence for the recommended architecture
- Two what-if results and the final recommendation with its overturning gate

Troubleshooting

Symptom	What to do
Local timings are much faster or slower than a classmate's	Expected: they are measured on each machine. Report yours with the hardware.
Every architecture fails	Change the plan (fewer takes, lower draft resolution, longer deadline) - never relax a gate to manufacture a winner.
cannot read the bench clip	Run from inside the lab folder; reinstall opencv-python if VideoCapture fails.
Our real prices differ	Good - put the current published rates in architectures.json and rerun.

Note: The same content, plus the full prompt pack, is in [labs/lab-12-cloud-local-production-architecture/README.md](#) and its PDF. Sources used in this lab: S12, S13, S14, S16, S17.

Preparing for the Assessment

Written Assessment WA(Q&A) — 13 questions, 60 minutes, open book

- One question per knowledge statement, K1 through K13, in that order.
- Every question is answerable from the slides. The answer key cites the slide.
- All questions are open-ended. There is no multiple choice anywhere in this assessment.
- Where a question asks you to NAME three of something, name three and say what each is for. A bare list is the minimum; the second clause is what a competent answer looks like.
- Where a question asks you to show code, show the call with its arguments — not pseudocode, and not just the function name.

Practical Performance (PP) — 5 tasks, 90 minutes, open book

- Task 1 covers A1; Task 2 covers A2 and A3; Task 3 covers A4; Task 4 covers A5 and A6; Task 5 covers A7 and A8.
- One continuous scenario — a different short film from The Last Ferry — runs through all five tasks, with its own resource files.
- Every task maps to a lab you have done. The model answer is that lab's procedure applied to the assessment scenario.
- Show your command AND your result. A result without the command that produced it cannot be assessed, and a command without a result has not been run.
- Where a task asks you to explain a difference, give the mechanism, not the outcome. 'The colour tracker failed' is not an answer; 'the tracker follows a hue histogram built on the first frame, so when the coat drifts from hue 23 to 13 the back-projection empties and the window jumps' is.

What to have open

- This Learner Guide.
- The slide deck PDF.
- Your own lab [out/](#) folders — your measurements are your working.
- The prompt packs in each lab's [PROMPTS.md](#).

Glossary

- **180-degree rule** — Keep the camera on one side of the line of action so screen directions stay consistent across cuts.
- **Animatic** — Storyboard keyframes timed to the voiceover; fixes timing before any generation credit is spent.
- **Aspect ratio** — Width : height of the frame. 16:9 master, 9:16 for Reels and Shorts, 4:5 and 1:1 for feeds.
- **Audio tag** — A bracketed direction in a voice script, e.g. [out of breath], that controls delivery line by line.
- **Bhattacharyya distance** — A bounded histogram distance where 0 means identical palettes. The look-bible gate in Lab 07 uses it.
- **C2PA Content Credentials** — Tamper-evident metadata recording a file's origin, tool and edit history.
- **CAMShift** — Continuously Adaptive Mean Shift: climbs a hue back-projection to track a subject and adapts the window size each frame.
- **Character reference sheet** — Front, side and back views plus close-ups of one character in one outfit — the feature specification every shot is checked against.
- **Continuity block** — The prompt text that must be identical in every shot: wardrobe, time of day, weather.
- **Diffusion transformer (DiT)** — A transformer that denoises spatiotemporal patches of a latent video over S steps.
- **Element** — A saved, named reference (character, location, prop) reused by name in every prompt.
- **Flicker index** — Mean frame-to-frame change in brightness inside one shot.
- **Generative expand (outpainting)** — Extending the canvas and generating the new area, e.g. 16:9 to 9:16.
- **Identity drift** — A character's features (face, wardrobe colour) changing across frames or shots.
- **Inpainting** — Regenerating only the masked region of an image.
- **IoU** — Intersection over union of two boxes. Detection matching also needs the right class, a stated threshold and one-to-one assignment.
- **Long-running operation** — An asynchronous API job: submit, receive an ID, poll or receive a webhook, then download.
- **LUT** — Lookup table: a point operation mapping each input value to an output value; how a grade is applied.

- **Multi-shot prompt** — One prompt listing Shot 1, Shot 2, Shot 3 in order, with dialogue in quotation marks.
- **ORB** — Oriented FAST keypoints with rotated BRIEF binary descriptors, matched with Hamming distance.
- **PSNR** — Peak signal-to-noise ratio: $10 \cdot \log_{10}(255^2 / \text{MSE})$. Rewards per-pixel accuracy, forgives blur.
- **Quantisation** — Storing weights at fewer bits; memory scales by $b/32$.
- **Reference-first** — Building character, location and prop references before any video generation, so the model has nothing left to guess.
- **Signed URL** — A time-limited link that lets a client upload or download one file from object storage.
- **SSIM** — Structural similarity: luminance, contrast and structure over local windows.
- **Start / end frame** — Images that fix where a generated shot begins and where it must land.
- **State variant** — The same location or prop in a different story state, made by editing the original ('change nothing else').
- **Temporal coherence** — Consistency ACROSS frames — the defining problem of AI video.
- **Timeline prompt** — A prompt with time-stamped beats, e.g. [0-2s] ... [2-5s] ...
- **Video extension / reference** — Passing the previous clip to the model so lighting, mood and motion carry across the cut.
- **Voiceover-first** — Generating and timing the voice before the pictures, then cutting the storyboard to it.

Source Register

Every external claim in the slides and in this guide traces to one of the following. The tutorials S6–S12 are public creator videos, several sponsored by the tools they demonstrate: they anchor techniques, not tool recommendations. No ebook pages or video frames are reproduced — each entry records the locator and the fact it anchors.

Ref	Source	Locator	What it anchors
S1	Course Proposal Tertiary Infotech CA- WSQ-2020-013290-v2 (SSG-approved)	Part 4 Section E, Curriculum Key Features table	Approved hour breakdown: 7.5 h classroom facilitation + 6 h practical + 2.5 h assessment = 16 h; per- LU instructional minutes (75 min CR each; practical 0/60/60/120/60/60); WA 60 min and PP 90 min.
S2	Assessment Plan_OpenCV_TGS-20 20505925_V1.0 (28 Dec 2020)	Assessment Duration table; WA and PP specification tables	WA(Q&A) 60 min covering K1-K13; PP 90 min covering A1-A8; assessor:candidate 1:3 to 1:10; open-book; C/NYC decision rule.

Ref	Source	Locator	What it anchors
S3	Competency Mapping.docx (TSC ICT-DIT-4022-1.1)	Course Mapping section and ELO/A-code matrix	Verbatim K1-K13 and A1-A8 statements; ELO-to-ability matrix (ELO1:A1 / ELO2:A2 / ELO3:A3,A4 / ELO4:A5 / ELO5:A6 / ELO6:A7,A8).
S4	Computer Vision Technology.docx (Skills Framework TSC extract)	TSC Proficiency Level 4 (ICT-DIT-4022-1.1) knowledge and abilities columns	TSC title, code, description and the authoritative knowledge/ability wording.
S5	Tertiary Courses public registration page and LMS-TMS course record	https://www.tertiarycourses.com.sg/wsqa-create-short-video-film-using-ai.html (checked 27 Sep 2026)	Registered title 'Create Short Video Film using AI', TGS code, the five delivery topics, the six learning outcomes, the course description (concept, storyline, script, characters, storyboard, text-to-image, image-to-video, text-to-video, cinematic shots, voiceover, music, sound effects, captions, final edit), 2-day/16-hour duration and target learners.
S6	Youri van Hofwegen, 'EXACTLY How to Make an AI Short Film (Full Workflow)', YouTube, 1 Jun 2026, youtube.com/watch?v=0v534yAyhgw	Full video, 16:49	Reference-first principle (stop giving the model freedom); three reference types — characters, locations, props; casting attributes (genre, budget, era, archetype, identity, physique, details, outfit); a second, damaged-state location made by editing the first ('change nothing else'); director controls (genre, camera movement, speed ramp, duration, aspect ratio, audio); per-character emotion and intensity; the multi-shot prompt framework with dialogue in quotation marks; video reference versus start/end frame (mood carries over); three generations for dense action shots; assembly in a timeline editor.

Ref	Source	Locator	What it anchors
S7	Mira AI, '6 AI Video Tools I'd Use To Make An AI Short Film', YouTube, 14 Jun 2026, youtube.com/watch?v=Wwh8vCqboxQ	Full video, 10:04	A multi-tool workflow: an LLM to plan the story shot by shot, a cinematic image model for character sheets (front, side, back, close-ups) and multi-angle environment grids, a design canvas for the storyboard with keyframe placeholders, a video model for motion, an AI music generator (instrumental, mood and tempo), and an editor; the six-point prompt framework (subject, action, environment, camera, style, constraints); timeline prompting; video extension to pass the previous clip; generations capped at about 15 seconds.
S8	Thomas Creates, 'How to Start Making AI Short Films in 2026', YouTube, 18 Sep 2026, youtube.com/watch?v=69_EJD4FEPg	Video description and chapters, 12:42	Consistent characters, props and locations; animating cinematic shots; carrying the same lighting and look between scenes; generated narration; assembling the film.
S9	Roboverse, 'The Complete Guide to Making Cinematic AI Videos (2026)', YouTube, 19 Jul 2026, youtube.com/watch?v=HMVSCEh72n4	Chapters, 10:30	Planning the story with an LLM; developing the narrative; creating consistent assets; staging the visual scenes; cinematic visual grading for a film look before animation; refining motion and prompts; generating clips; post-production and finishing.
S10	Youri van Hofwegen, 'How to Make Your First AI Movie (Full Guide)', YouTube, 11 Jun 2026, youtube.com/watch?v=fs5S867VQzg	Video description, 17:03	Generating the story; building consistent characters and locations; JSON prompting; asset management; video references for connected scenes; editing; fixing voice continuity across clips.

Ref	Source	Locator	What it anchors
S11	ElevenLabs, 'How to Make Your First AI Short Film (Full Tutorial)', YouTube, 9 Aug 2026, youtube.com/watch?v=47DH_VUa67A	Chapters, 31:13	Explore cheaply with faster image models before committing credits; prompt structure across style, subject, objects and negative prompting; a character reference sheet; generate the voiceover FIRST and build the storyboard to match it; direct delivery line by line with audio tags; still frames before video; start and end frames; generate at lower resolution and upscale only the takes you keep; background music; sound effects; edit and trim on a timeline; regenerate with a new character.
S12	Sanky, 'How I Made an AI Short Drama in under 10 Minutes with ZooClaw + Seedance2', YouTube, 2 Jun 2026, youtube.com/watch?v=q8z5iK5Pm_s	Chapters and description, 7:47	An all-in-one drama agent: upload one photo as the protagonist, describe the story, and the agent produces character-consistent shots, voiceover, music, storyboard and video; the prompt formula (premise, main character, scene style, numbered story beats, 'include voiceover, music and subtitles'); trade-off against node-based workflows.
S13	M. Tschochohei and F. Schenker, 'Multimodal Generative AI in the Enterprise: From Pixels to Profit', Packt, June 2026, ISBN 978-1-80611-167-1	Ch.5 pp.56-75; Ch.7 pp.106-119; Ch.8 pp.127-143	VAE, GAN and diffusion mechanisms and failure modes; prompt structure and subject lock; binary masks (white = editable), inpainting, outpainting and mask-free editing; spatial versus temporal coherence, 3-D U-Nets and diffusion transformers over spatiotemporal patches; long-running operation polling; responsible-AI risks (voice cloning, deepfakes, likeness) and transparency (C2PA Content Credentials, invisible watermarking, model cards).

Ref	Source	Locator	What it anchors
S14	A. Mewada, M. A. Ansari, S. Ahmad and N. Singh (eds), 'AI-Generated Image and Video Synthesis', IEEE Press / Wiley, 2026, ISBN 9781394403110	Ch.1 pp.1-11; Ch.3 pp.35-44; Ch.7 pp.94-100	Frame-based versus temporal video generation and the temporal-consistency challenge; the sensors-to-synthesis layered architecture and split computing between edge and cloud; evaluation (FID, KID, LPIPS, CLIP faithfulness, FVD plus optical-flow agreement); memory = parameters x precision, quantisation to b bits scales by b/32, diffusion latency proportional to S steps (S x F for video).
S15	Google AI for Developers — Gemini API image generation documentation	https://ai.google.dev/gemini-api/docs/image-generation (verified 6 Sep 2026)	Native image models for text-to-image and conversational image editing; supported aspect ratios including 16:9, 9:16, 4:5 and 21:9; output sizes up to 4K by model; every generated image carries a SynthID watermark.
S16	Google AI for Developers — Veo video generation documentation	https://ai.google.dev/gemini-api/docs/veo (verified 6 Sep 2026)	Veo 3.1 model family with native audio; 4, 6 or 8 second clips; 16:9 or 9:16; 720p, 1080p or 4k; up to three reference images on supported models; image-to-video; first and last frame control; clip extension; asynchronous long-running operation that the client polls.

Ref	Source	Locator	What it anchors
S17	OpenCV 4.13 Python package and FFmpeg, verified locally on the delivery image	cv2 4.13.0 (opencv-python 4.13.0.92); ffmpeg/ffprobe optional	Every lab command runs on the base opencv-python wheel plus NumPy: VideoCapture / VideoWriter, cvtColor, calcHist/compareHist, Canny, ORB_create + BFMatcher, HuMoments, inpaint, CLAHE, filter2D, fastNlMeansDenoisingColored, Haar cascades from cv2.data.harcascades, HOGDescriptor_getDefaultPeopleDetector, CamShift and calcOpticalFlowFarneback. SSIM is implemented in NumPy because cv2.quality is contrib-only. FFmpeg is used only when present, to mux audio into the final export.